Chronic Kidney Disease Prediction
Project Report

Ramzy Ashraf - 21069835
Omar Hamdi- 21065841
Hassan Mahmoud - 21071135
Omar Elsayed - 21069933
Noran Hesham - 21061141
Renad Mahmoud - 21063392

Digital Egypt Pioneers Initiative
Round 2

May 10, 2025

Contents

1 Introduction to the project

Chronic Kidney Disease (CKD) is a long-term medical condition characterized by a gradual loss of kidney function, potentially leading to kidney failure if not diagnosed and treated in its early stages. According to the World Health Organization, CKD is a growing global health concern, affecting millions of individuals and placing a heavy burden on healthcare systems. Early detection is crucial, as timely medical intervention can slow the progression of the disease and improve patient outcomes. However, early-stage CKD often goes undetected due to subtle or non-specific symptoms, making it challenging for traditional diagnosis methods to catch it in time.

This project aims to develop a machine learning-based predictive system for the early detection of Chronic Kidney Disease. The goal is to build an accurate and reliable model that can analyze clinical data and classify whether a patient is likely to have CKD. The system is intended to serve as a decision-support tool for healthcare providers, aiding them in identifying high-risk patients and initiating further diagnostic procedures or treatment.

To achieve this, a supervised learning approach was employed using a publicly available CKD dataset from the Chronic Kidney Disease Dataset. The dataset contains clinical features such as blood pressure, serum creatinine, hemoglobin levels, and other relevant indicators. After preprocessing the data—handling missing values, encoding categorical variables, and scaling numerical features—multiple classification algorithms were evaluated. The best-performing model was selected based on accuracy, precision, recall, and F1-score metrics.

The final model was deployed using Streamlit, an open-source Python library that allows rapid development of interactive web applications. The deployed application provides a simple and intuitive interface where users can input clinical data and receive a real-time prediction on CKD risk. This tool demonstrates the potential of integrating machine learning with digital health technologies to enhance disease screening and preventive care.

2 Data Collection, Exploration and Processing

2.1 Data Importing, Exploration and Understanding

The notebook begins by importing essential libraries such as pandas, matplotlib, pyplot, and seaborn, which are widely used for data handling and visualization. The Chronic Kidney Disease data.csv dataset is loaded using pandas.read function with the first column set as the index. Initial exploration includes displaying the first few rows of the dataset using head(), checking for missing values with isnull().sum(), and assessing the structure and summary statistics of the data via info() and describe() functions. A diagnosis class imbalance is observed, which is flagged for later handling using the SMOTE technique.

In the data cleaning phase, the notebook drops several columns considered insignificant for predictive modeling, such as EducationLevel, SocioeconomicStatus, DoctorInCharge, and QualityOfLifeScore.

2.2 EDA: Exploratory Data Analysis

The exploratory data analysis (EDA) section is thorough, beginning with univariate analysis using histograms and count plots to visualize distributions of individual features such as GFR and Smoking. Bivariate analysis follows, utilizing boxplots and scatterplots to examine the relationships between Diagnosis and features like GFR and Age. A pairplot is used for multivariate visualization across key features, including AlcoholConsumption, HbA1c, and ProteinInUrine, highlighting differences by diagnosis class.

Additionally, a correlation heatmap is plotted for a large subset of features, including biochemical markers and blood pressure measurements, to understand interdependencies and identify potential multicollinearity. Further EDA examines patterns such as the average age of CKD diagnosis and its distribution, as well as visual investigations into whether ethnicity or lifestyle factors (e.g., smoking and alcohol) influence CKD prevalence.

2.3 Data Visualization Techniques and Their Importance

We employed a variety of data visualization techniques to analyze the Chronic Kidney Disease (CKD) dataset. These visual tools provide intuitive insights into the structure of the data, identify potential patterns and anomalies, and help guide decisions in data preprocessing and modeling. Below is a detailed account of the visualization methods used:

2.3.1 1. Histogram with KDE (Kernel Density Estimation)

```
sns.histplot(kidney['GFR'], kde=True)
```

Purpose: This plot visualizes the distribution of the `GFR` (Glomerular Filtration Rate), which is a critical metric for kidney function.

Advantages:

- Combines histogram (discrete bin frequencies) with a smoothed KDE curve for better distribution estimation.

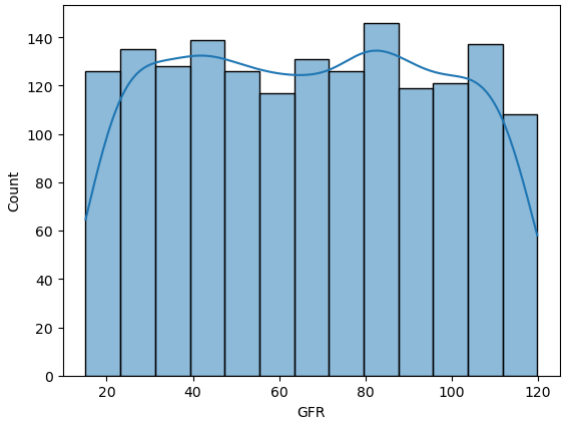

Figure 1: A Histogram with KDE visualization example

2.3.2 2. Count Plot

```
sns.countplot(data=kidney, x='Smoking')
```

Purpose: Displays the number of instances for each category of the `Smoking` feature.
Advantages:

- Ideal for visualizing categorical data.
- Highlights class imbalance.
- Assists in detecting patterns relevant to CKD diagnosis.

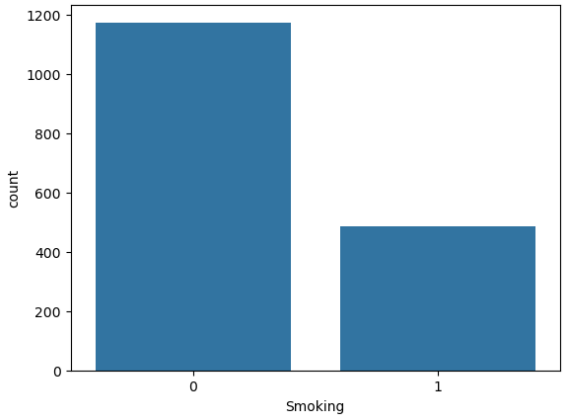

Figure 2: A Count Plot as an illustration

2.3.3 3. Boxplot

```
sns.boxplot(data=kidney, x='Diagnosis', y='GFR')
sns.boxplot(data=kidney, x='Diagnosis', y='Age')
```

Purpose: Compares numerical distributions (e.g., `GFR`, `Age`) across different diagnosis groups.

Advantages:

- Clearly shows medians, quartiles, and outliers.
- Useful for evaluating differences in distributions between classes.
- Helps detect feature importance and influence on CKD.

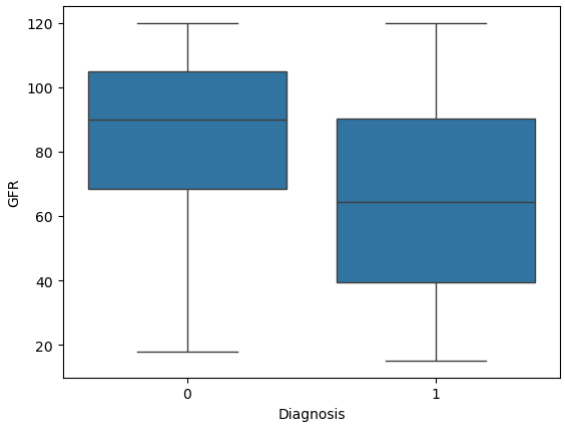

Figure 3: A Boxplot of the type of Diagnosis and GFR as a visualization example

2.3.4 4. Pair Plot

```
sns.pairplot(kidney, vars=features, hue="Diagnosis", diag_kind="kde", markers=["o", "s"])
```

Purpose: Creates a matrix of scatter plots and KDE plots to explore relationships between multiple features, colored by `Diagnosis`.

Advantages:

- Highlights correlation and clustering patterns between variables.
- Facilitates multi-feature analysis.
- Aids in feature selection for model input.

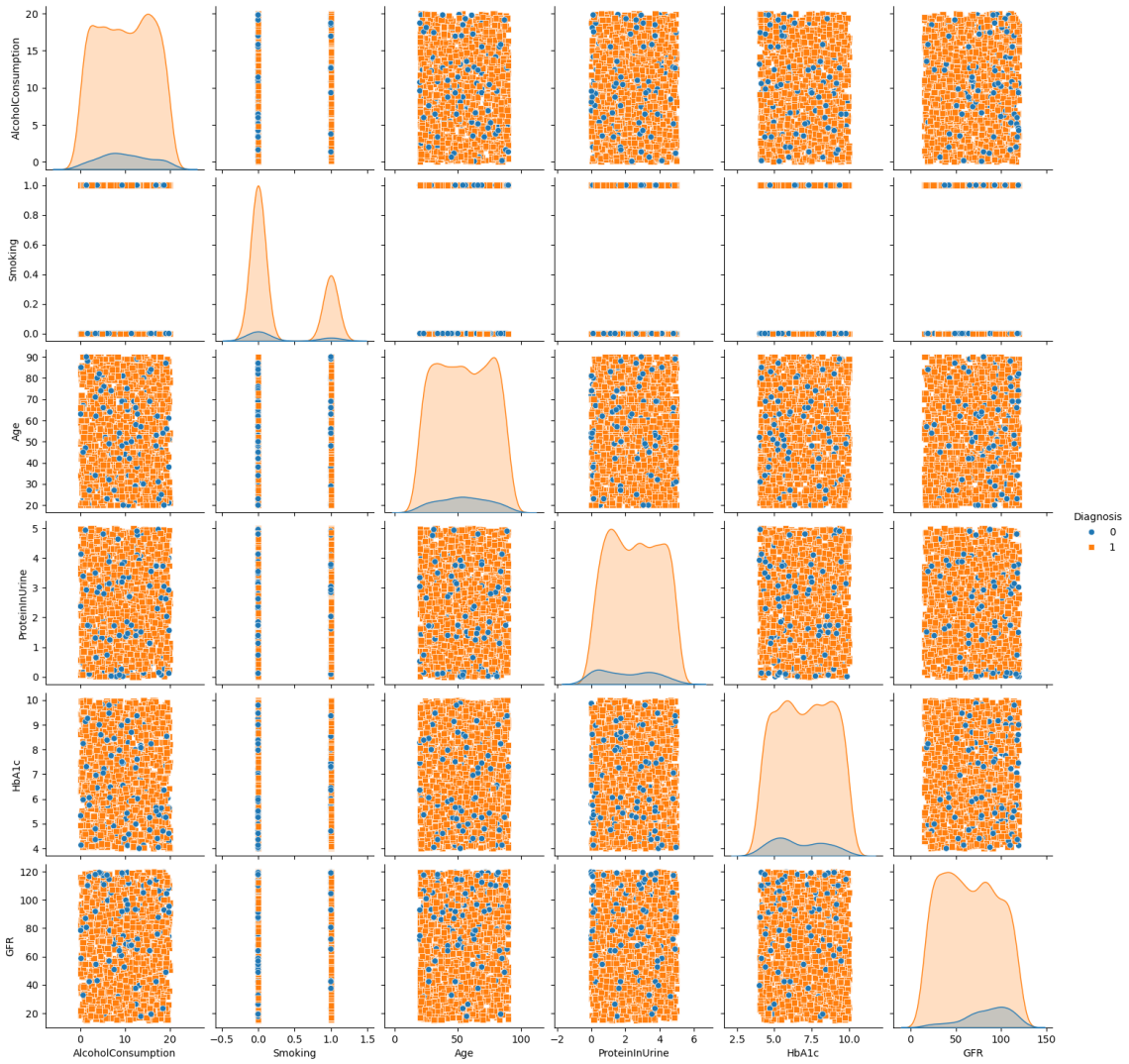

Figure 4: A pair plot for illustrative purposes for certain variables

2.3.5 5. Heatmap of Correlation Matrix

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
```

Purpose: Visualizes pairwise Pearson correlation coefficients among numerical features.
Advantages:

- Identifies strongly correlated features (positive or negative).
- Useful for diagnosing multicollinearity.
- Guides dimensionality reduction and feature engineering.

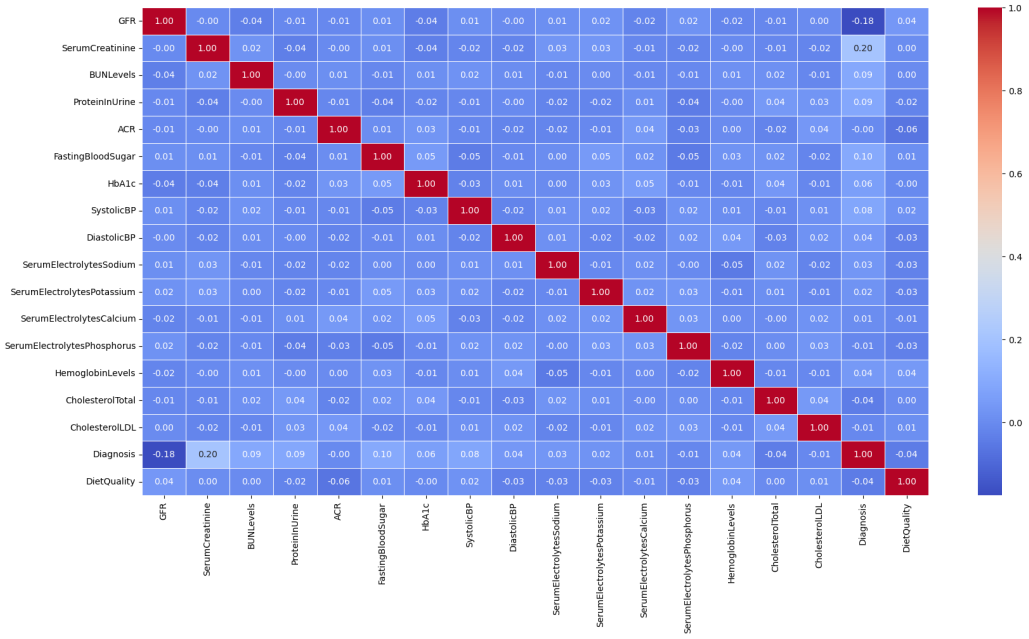

Figure 5: A heat map tracing the correlation between a set of variables

2.3.6 6. Scatter Plot

```
sns.scatterplot(x='Age', y='GFR', hue='Diagnosis', data=kidney)
```

Purpose: Plots the relationship between `Age` and `GFR`, distinguished by CKD diagnosis class.
Advantages:

- Shows the distribution and trend between two continuous variables.
- Reveals potential class separability and interaction effects.
- Supports hypothesis generation for clinical patterns in CKD.

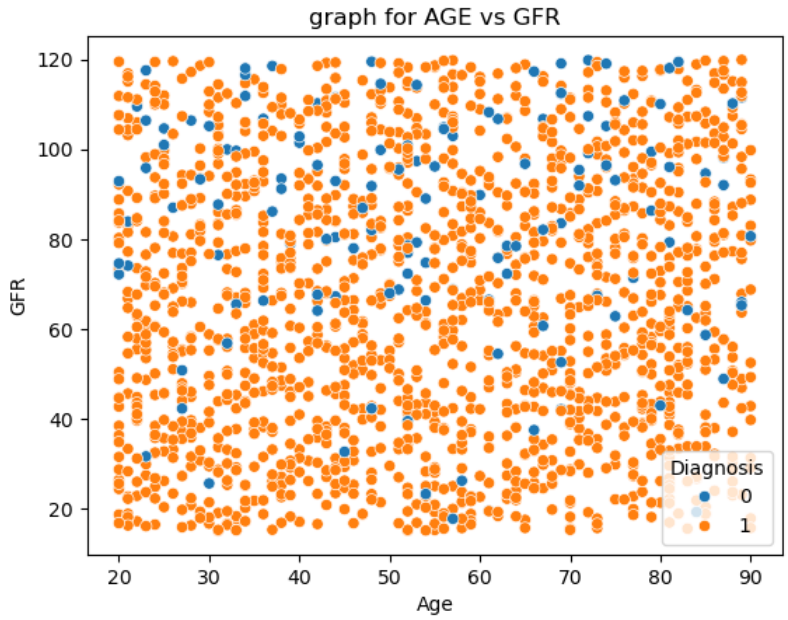

Figure 6: A scatter plot for the GFR and Age intervals in the CKD dataset

These visualization techniques play a crucial role in making the data interpretable and model-ready, bridging the gap between raw numerical data and informed decision-making.

Below are screenshots of the written analysis-based answers to the questions posed in the project about the correlation of different data:

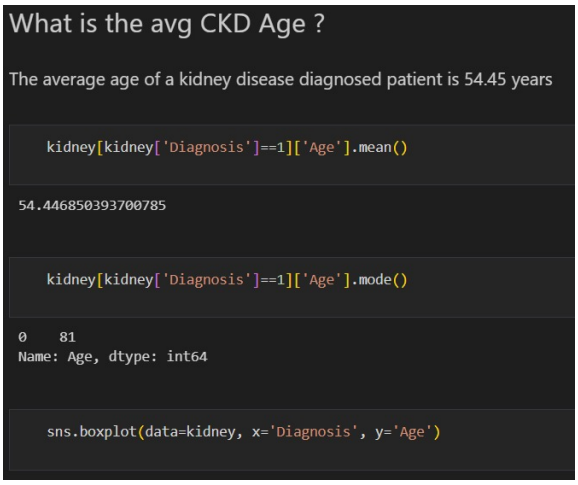

Figure 7: Answering "What is the average CKD patient age?"

Figure 8: Answering "Does Ethnicity correlate with CKD patients?"

```
sns.countplot(x='Ethnicity', hue='Diagnosis', data=kidney)
plt.title("CKD Diagnosis vs Ethnicity")
plt.xlabel("Ethnicity")
plt.ylabel("Number of Patients")
plt.legend(title="CKD Diagnosis")
plt.show()
```

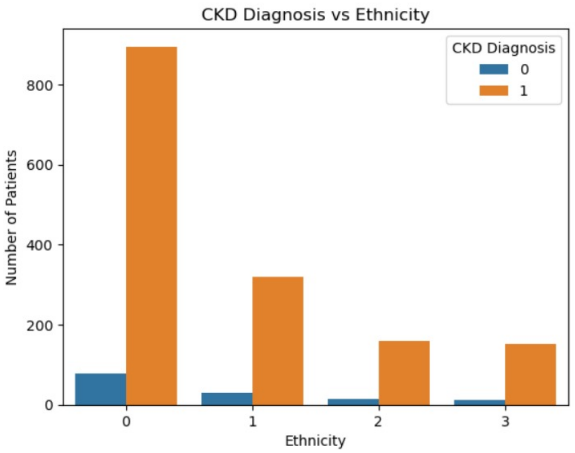

Figure 9: Answering "Does Ethnicity correlate with CKD patients?"

Does FamilyHistoryKidneyDisease affects CKD ?

```
from scipy.stats import chi2_contingency
```

```
kidney['FamilyHistoryKidneyDisease']
```

```
PatientID
1       0
2       1
3       0
4       0
5       0
       ..
1655    0
1656    0
1657    0
1658    0
1659    1
Name: FamilyHistoryKidneyDisease, Length: 1659, dtype: int64
```

Figure 10: Answering "Does Family History affect CKD patients?"

Does Gender affects CKD ?

```
sns.countplot(x='Gender', hue='Diagnosis', data=kidney)
plt.title("CKD Diagnosis vs Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Patients")
plt.legend(title="CKD Diagnosis")
plt.show()
```

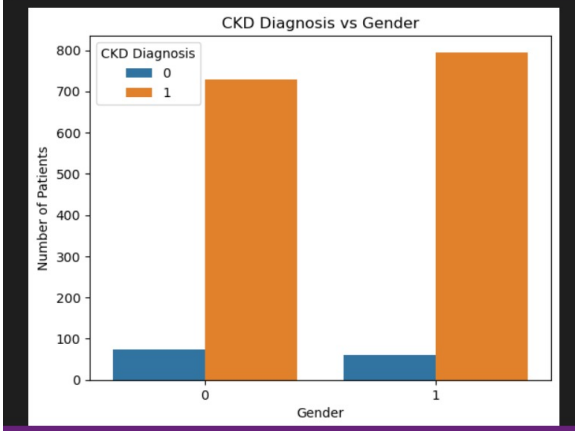

Figure 11: Answering "Does Gender affect CKD patients?"

```
# Filter the data for individuals with Diagnosis = 1 (CKD)
ckd_data = kidney[kidney['Diagnosis'] == 1]

# Visualize the percentage distribution for numerical features
numerical_features = ckd_data.select_dtypes(include=['int64', 'float64']).columns

# Plot histograms for each numerical feature
for feature in numerical_features:
    plt.figure(figsize=(8, 6))
    sns.histplot(ckd_data[feature], kde=True, bins=20)
    plt.title(f'Distribution of {feature} when Diagnosis = 1')
    plt.xlabel(feature)
    plt.ylabel('Frequency')
    plt.show()

# Visualize the percentage distribution for categorical features
categorical_features = ckd_data.select_dtypes(include=['object', 'category']).columns

# Plot bar plots for each categorical feature
for feature in categorical_features:
    plt.figure(figsize=(8, 6))
    sns.countplot(x=feature, data=ckd_data, palette='Set2')
    plt.title(f'Distribution of {feature} when Diagnosis = 1')
    plt.xlabel(feature)
    plt.ylabel('Count')
    plt.show()
```

Figure 12: All the distribution visualizations of dependent variables for-loop

3 Model Development and Optimization

The process of developing and optimizing a machine learning model for diagnosing Chronic Kidney Disease (CKD) involves multiple critical steps. Each stage ensures the model is accurate, fair, and robust in predicting outcomes on unseen data. Below, we detail the components typically involved in this pipeline:

1. Data Splitting

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Purpose: Splits the dataset into training and testing sets, usually using an 80-20 ratio.
Importance:

- Prevents overfitting by ensuring the model is validated on unseen data.
- Provides a realistic evaluation of model generalization performance.

```
# Filter the data for individuals with Diagnosis = 1 (CKD)
ckd_data = kidney[kidney['Diagnosis'] == 1]

# Visualize the percentage distribution for numerical features
numerical_features = ckd_data.select_dtypes(include=['int64', 'float64']).columns

# Plot histograms for each numerical feature
for feature in numerical_features:
    plt.figure(figsize=(8, 6))
    sns.histplot(ckd_data[feature], kde=True, bins=20)
    plt.title(f'Distribution of {feature} when Diagnosis = 1')
    plt.xlabel(feature)
    plt.ylabel('Frequency')
    plt.show()

# Visualize the percentage distribution for categorical features
categorical_features = ckd_data.select_dtypes(include=['object', 'category']).columns

# Plot bar plots for each categorical feature
for feature in categorical_features:
    plt.figure(figsize=(8, 6))
    sns.countplot(x=feature, data=ckd_data, palette='Set2')
    plt.title(f'Distribution of {feature} when Diagnosis = 1')
    plt.xlabel(feature)
    plt.ylabel('Count')
    plt.show()
```

Figure 13: Splitting code

2. Handling Class Imbalance with SMOTE

```
from imblearn.over_sampling import SMOTE
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)
```

Purpose: Balances the training data when one class (e.g., CKD-positive) is underrepresented.
Importance:

- Prevents the model from being biased toward the majority class.
- Improves recall and F1-score for the minority class.

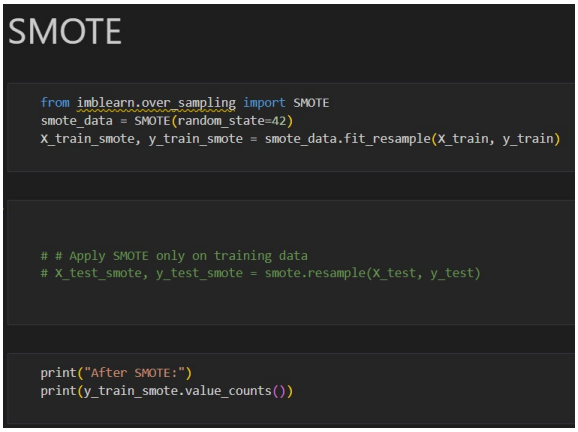

Figure 14: Using the SMOTE technique for data interpolation

3. Model Selection and Training

Various algorithms are typically evaluated:

- Logistic Regression
- Decision Trees
- Random Forest
- K-Nearest Neighbors
- Support Vector Machine (SVM)
- XGBoost or LightGBM

Example:

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_res, y_res)
```

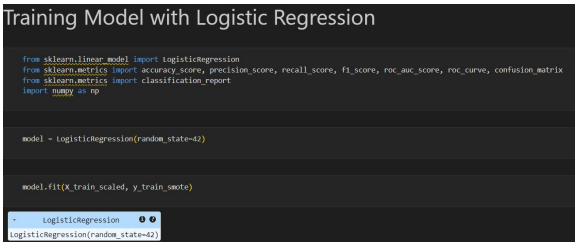

Figure 15: Training model with Logistic regression

Purpose: Identifies the most suitable model for the dataset.
Importance:

- Different models capture different patterns in data.
- Testing multiple models allows performance comparison.

4. Model Evaluation

```
from sklearn.metrics import classification_report, confusion_matrix
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

Metrics Used:

- Accuracy
- Precision and Recall
- F1 Score
- Confusion Matrix
- ROC-AUC Score

Importance:

- Evaluation beyond accuracy is essential in medical diagnosis.
- Recall is especially critical to minimize false negatives.

Below is the Model evaluation function:

```
def evaluate_model(X, y, dataset_name):
y_pred = model.predict(X)
y_proba = model.predict_proba(X)[:, 1]  # Probability for positive class (for ROC-AUC)

# Metrics
accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred, average='weighted')
recall = recall_score(y, y_pred, average='weighted')
f1 = f1_score(y, y_pred, average='weighted')
roc_auc = roc_auc_score(y, y_proba, multi_class='ovr') if len(np.unique(y)) > 2 else
roc_auc_score(y, y_proba)

print(f"\nEvaluation on {dataset_name}:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

```
    print("\nClassification Report:")
    print(classification_report(y, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title(f'Confusion Matrix - {dataset_name}')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

    if len(np.unique(y)) == 2:
        fpr, tpr, _ = roc_curve(y, y_proba)
        plt.figure(figsize=(6, 4))
        plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.4f})')
        plt.plot([0, 1], [0, 1], 'k--')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC Curve - {dataset_name}')
        plt.legend(loc='best')
        plt.show()

# Evaluate on validation and test sets
evaluate_model(X_val_scaled, y_val, "Validation Set")
evaluate_model(X_test_scaled, y_test, "Test Set")
```

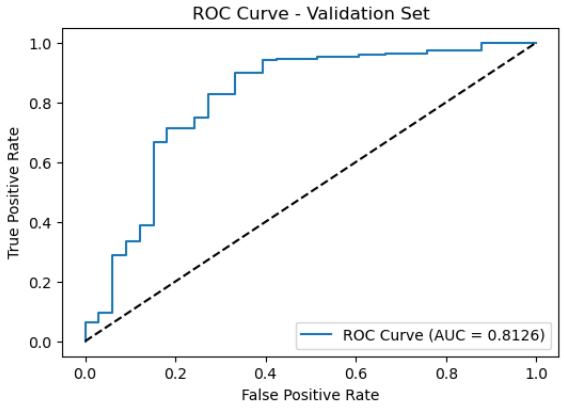

Figure 16: ROC Curve- Validation set as an example

5. Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators': [100, 200], 'max_depth': [5, 10]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid.fit(X_res, y_res)
```

Purpose: Searches for the optimal combination of model parameters.
Importance:

- Improves model accuracy and generalization.
- Helps avoid overfitting or underfitting.

6. Feature Importance Analysis

```
importances = model.feature_importances_
sns.barplot(x=importances, y=feature_names)
```

Purpose: Identifies which features most influence the model's predictions.
Importance:

- Improves the interpretability of the model.
- Guides future feature selection or medical investigation.

7. Final Validation

```
final_preds = grid.predict(X_test)
print(confusion_matrix(y_test, final_preds))
```

Purpose: Tests the final tuned model on the test set.
Importance:

- Confirms model robustness and real-world applicability.
- Ensures the improvements are not due to overfitting.

This structured approach to model development and optimization ensures that the final CKD diagnosis model is both scientifically rigorous and clinically valuable.

The models used and evaluated

4 MLOps, Deployment and Monitoring

After a model is trained and optimized, it is essential to operationalize it through MLOps principles to ensure reliable deployment, reproducibility, and real-time monitoring. Below is a structured description of how this is typically approached:

1. Model Serialization and Export

Purpose: Save the trained model in a binary file for future use or deployment.
Importance:

- Enables model reuse without retraining.
- Supports deployment in different environments.

It was made by a pickle and Streamlit.

4.1 2. API Development for Model Serving

A lightweight REST API can be created using Flask:

```
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load('ckd_model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    input_data = request.get_json(force=True)
    prediction = model.predict([list(input_data.values())])
    return jsonify({'prediction': prediction[0]})
```

Purpose: Exposes the model to external systems via HTTP.
Importance:

- Allows integration with hospital systems, mobile apps, or dashboards.
- Makes the model accessible in real-time through a secure endpoint.

4.2 3. Containerization with Docker

A Dockerfile can be used to containerize the application:

```
FROM python:3.10
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Purpose: Encapsulates the environment and dependencies into a portable image.
Importance:

- Guarantees consistent runtime across different machines or cloud providers.
- Simplifies deployment and scaling.

4.3 4. Continuous Integration and Deployment (CI/CD)

Using platforms like GitHub Actions or GitLab CI, workflows can be automated:

- Run unit tests on each push.
- Lint and validate code formatting.
- Automatically build Docker images and push to a container registry.
- Deploy the container to cloud platforms (e.g., Heroku, AWS, Azure).

Importance:

- Streamlines updates and deployment.
- Reduces human error through automation.
- Ensures reproducibility and traceability.

4.4 5. Monitoring and Logging

For post-deployment monitoring, tools such as Prometheus, Grafana, or custom logging can be integrated:

- Track prediction frequency and latency.
- Monitor model drift (change in input data patterns).
- Collect error rates and user feedback for continuous improvement.

Importance:

- Maintains system health and reliability.
- Detects anomalies and allows rapid rollback if needed.
- Ensures model fairness and performance over time.

4.5 6. Model Versioning and Governance

Using tools like MLflow or DVC, the following are managed:

- Dataset versions
- Model metrics
- Experiment tracking

Importance:

- Supports auditability and compliance in healthcare settings.
- Facilitates reproducible research and collaboration.

By following MLOps practices, the CKD prediction model moves from a static notebook to a scalable, monitored, and deployable system, ready for integration into real-world clinical environments.